



Université du Québec
à Chicoutimi

TP2 : Déploiement d'une application web à trois niveaux

UQAC - Université du Québec à Chicoutimi

Diplome - Maîtrise en informatique (Cybersécurité)

8INF876 - Conception et architecture des systèmes d'infonuagique

Par Sofiane EL NAGGAR - ELNS16100200

Matéo HANY - HANM22100200

Aubry TONNERRE - TONA19040200

Année universitaire : 2024-2025

1 Introduction

L'objectif de ce projet est de déployer une application web structurée en trois niveaux, intégrant une base de données mysql, une interface utilisateur (frontend) et une couche de traitement (backend). Ce déploiement est réalisé grâce à la conteneurisation, en utilisant des services et des volumes persistants dans un environnement orchestré par Kubernetes, le tout configuré sur Minikube.

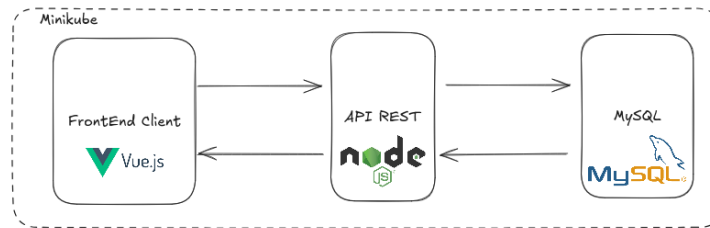


Figure 1: Architecture trois niveaux

Le projet se base sur une application de calcul de l'IMC, développée lors du TP1. Le but est de démontrer les avantages de la conteneurisation et de l'orchestration dans la gestion et le déploiement d'applications.

2 Pratique

2.0.1 Etape 1 - Prérequis

Afin de pouvoir monter notre architecture, nous devons installer deux composants en plus de docker: `kubectl` et `Minikube`.

Kubernetes est un orchestrateur de conteneurs open-source permettant d'automatiser le déploiement, la gestion et la mise à l'échelle des applications conteneurisées. Grâce à l'outil en ligne de commande `kubectl`, les opérations de mise à l'échelle, d'équilibrage de charge et de résilience des services sont facilitées. Il s'installe à l'aide des commandes [1]:

```
1 sudo apt-get update && \
2 sudo apt-get install -y apt-transport-https ca-certificates curl gnupg && \
3 curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg && \
4 sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg && \
5 echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | \
6 sudo tee /etc/apt/sources.list.d/kubernetes.list && \
7 sudo chmod 644 /etc/apt/sources.list.d/kubernetes.list && \
8 sudo apt-get update && \
9 sudo apt-get install -y kubectl
```

Listing 1: Installation de kubectl

Minikube est un outil permettant de déployer un cluster Kubernetes local, conçu pour les tests et le développement. En reproduisant un environnement de production sur une machine locale, il facilite la vérification de la configuration et des interactions des Pods avant un déploiement à plus grande échelle. Il s'installe avec ces commandes [2]:

```
1 curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
2 sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64
```

Listing 2: Installation de Minikube

Nous utilisons la commande `minikube start` pour démarrer une instance de minikube que nous utiliserons par la suite pour déployer l'application.

2.0.2 Etape 2 - Déployer une base de données

L'objectif de cette étape est de mettre en place une base de données qui stockera plusieurs informations, notamment, les informations des utilisateurs ainsi que leur donnée d'IMC.

Le déploiement commence par la création d'un fichier de déploiement permettant de définir la configuration de la base de données MySQL de manière déclarative. Ce fichier inclut les spécifications essentielles au déploiement et à la gestion du conteneur, notamment le port d'accès à la base de données, fixé ici au port 3306 du conteneur. Des variables d'environnement sont également définies afin de paramétrer la base de données, incluant des informations comme le nom de la base, l'utilisateur, et le mot de passe requis pour l'accès. Le fichier de déploiement spécifie aussi le nombre de réplicas à maintenir. Cette configuration assure la disponibilité de la base de données : en cas d'interruption d'un conteneur, un autre prendra immédiatement en charge les requêtes, garantissant la continuité du service pendant la relance du conteneur initial.

Nous utilisons la commande suivante pour déployer notre service.

```
1 kubectl apply -f mysql-deployment.yaml
```

Cette commande déploie dans Minikube un Pod possédant une adresse IP unique et hébergeant la base de données MySQL accessible via le port 3306. La commande automatise le déploiement des ressources définies dans le fichier YAML, garantissant un déploiement structuré et reproductible. A ce moment là, la base de données n'est pas encore disponible par les autres pods.

Le déploiement du service permet aux autres Pods de contacter la base de données. Un nouveau fichier YAML est utilisé pour spécifier le port et le Pod à exposer. Il est possible de configurer ce port pour un accès restreint à l'intérieur de Minikube ou pour un accès externe, selon les besoins. Dans ce cas, pour faciliter les tests, la base de données est exposée à l'extérieur en définissant l'attribut `type` sur `NodePort`. Si un accès limité aux Pods internes au cluster avait été souhaité, l'attribut `ClusterIP` aurait été utilisé.

La même commande est exécutée pour déployer ce fichier de service.

À ce stade du projet, un Pod MySQL a été déployé ainsi qu'un service exposant le port 3306 du Pod sur le port 30006 de la machine locale. Un réplica a été déployé en plus pour la redondance.

2.0.3 Etape 3 - Déployer une application backend

L'étape suivante consiste à déployer le backend de l'application, qui correspond à l'API REST. Un fichier `Dockerfile` est défini pour exposer les ports de l'application et configurer les variables d'environnement nécessaires à l'accès à l'API. Cette configuration est essentielle pour permettre au frontend de se connecter au backend et pour indiquer à l'API l'adresse et le port à utiliser pour communiquer avec la base de données.

Comme pour la base de données, un fichier de déploiement et un fichier de service sont créés. Le déploiement permet de générer l'image Docker, de définir les variables d'environnement et de spécifier le port d'accès, tandis que le service garantit l'accès au Pod correspondant. Le backend est ensuite déployé dans Minikube avec la commande `kubectl apply -f backend-déploiement.yml`, suivie de la commande équivalente pour le fichier de service.

À ce stade du projet, deux Pods, deux services et deux répliques ont été déployés. Le service du backend expose le port 3000 du Pod sur le port 30001 de la machine locale.

2.0.4 Etape 4 - Volumes Persistants

Une fois la base de données et le backend définis, un stockage permanent est mis en place pour permettre à MySQL de disposer d'un espace mémoire dédié au stockage des données IMC. Cela garantit que, même en cas d'interruption du service, les données seront conservées et récupérables lors de la relance.

Un fichier `mysql-pvc.yaml` est créé pour définir ce volume persistant, puis déployé dans Minikube. Cependant, le fichier `mysql-deployment.yaml` ne faisant pas initialement référence à ce volume, est modifié pour spécifier que les données doivent être enregistrées dans le volume `mysql-pvc`.

Après le déploiement et des tests effectués avec Postman, il est possible d'écrire des données dans la base via l'API et de les récupérer après l'extinction et le redémarrage du Pod MySQL. Cette fonctionnalité est essentielle dans les systèmes distribués. Lors de la mise à jour d'un module ou du redémarrage d'un service, certaines ressources, comme la base de données, doivent être redémarrées. Sans un stockage persistant, une interruption risquerait de provoquer une perte de données non sauvegardées, ce qui serait critique pour la fiabilité du système.

2.0.5 Etape 5 - Déployer une application frontend

Un fichier de déploiement et un fichier de service sont également définis pour le frontend, avec une configuration des ports permettant d'y accéder. Cette étape suit le même processus que pour les composants précédents, sans introduire d'éléments nouveaux.

2.0.6 Étape 6 - Accéder à l'application

Une fois tous les composants du projet déployés, l'accès au frontend peut être testé. La commande suivante est utilisée :

```
1 minikube service frontend-vue-app --url
```

Cette commande génère une adresse locale, par exemple `http://192.168.49.2:30000/`, permettant d'accéder au frontend. Dans le cas d'un déploiement sur un réseau, une URL accessible serait fournie, rendant l'application utilisable via un navigateur ou d'autres outils appropriés.

2.0.7 Étape 7 - Nettoyage

Dans le cadre de la maintenance ou des mises à jour, il est parfois nécessaire de supprimer certains Pods et de relancer leur déploiement. Cela permet d'intégrer les modifications apportées aux configurations ou aux images Docker utilisées. Pour effectuer cette opération, les commandes suivantes sont utilisées :

```
1 kubectl delete -f backend-deployment.yml
2 kubectl delete -f backend-service.yaml
3 kubectl delete -f frontend-deployment.yaml
4 kubectl delete -f frontend-service.yaml
5 kubectl delete -f mysql-service.yaml
6 kubectl delete -f mysql-deployment.yaml
7 kubectl delete -f mysql-pvc.yaml
```

Cette série de commandes supprime les déploiements, services et volumes correspondants. Dans le contexte des systèmes cloud et distribués, cette approche est essentielle pour assurer une flexibilité et une adaptabilité rapides. Les mises à jour de fonctionnalités ou de sécurité nécessitent souvent un redéploiement des ressources pour appliquer les changements sans perturber les autres composants du système. Grâce à Kubernetes, cette gestion est automatisée et uniformisée, réduisant les risques de erreurs humaines lors des modifications.

3 Conclusion

Le projet a permis de mettre en oeuvre une application web à trois niveaux, intégrant une base de données MySQL, une API REST pour le backend et un frontend accessible via Minikube. En utilisant des outils tels que Docker, Kubernetes et Minikube, les concepts de conteneurisation, d'orchestration et de persistance des données ont été appliqués.

Les déploiements des différents composants ont démontré l'importance de l'automatisation dans la gestion des systèmes distribués. L'utilisation de volumes persistants a garanti la conservation des données, même en cas d'interruption des services, renforçant ainsi la fiabilité du système. De plus, les services exposés ont facilité l'interaction entre les Pods et permis l'accès au frontend à partir d'une machine locale ou d'un réseau.

Ce projet illustre l'efficacité et la modularité des technologies d'infonuagique dans le déploiement d'applications évolutives et résilientes. Les connaissances acquises ouvrent des perspectives d'amélioration pour des projets de plus grande envergure, notamment en termes de scalabilité, de sécurité et d'intégration continue.

Bibliographie

- [1] "Install and set up kubectl on linux," Kubernetes. Section: docs. (), [Online]. Available: <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/> (visited on 11/13/2024).
- [2] "Minikube start," minikube. (), [Online]. Available: <https://minikube.sigs.k8s.io/docs/start/> (visited on 11/13/2024).